



CMPT 413/713: Natural Language Processing

Classification

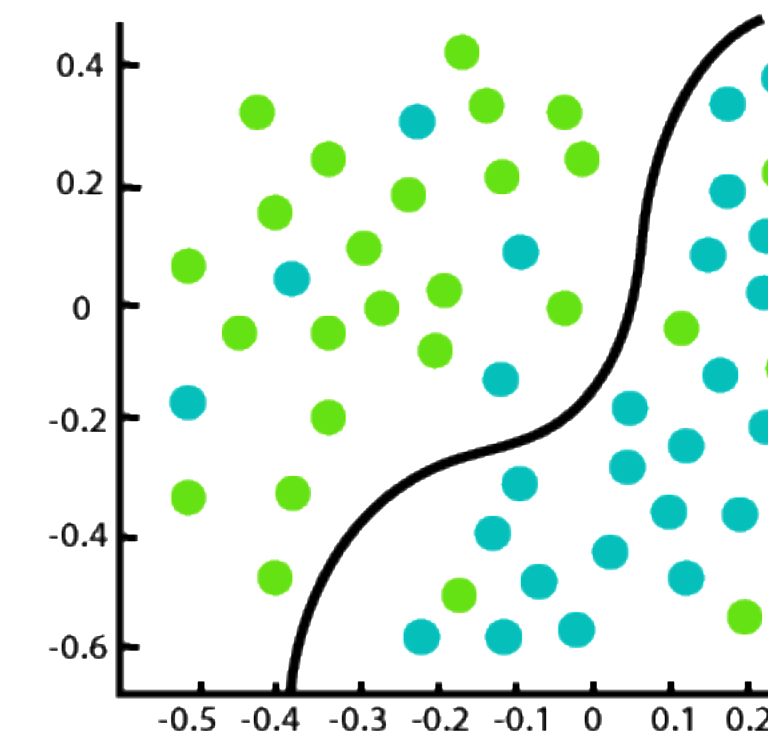
Zhiyun (Richard) Peng

2026-09-16

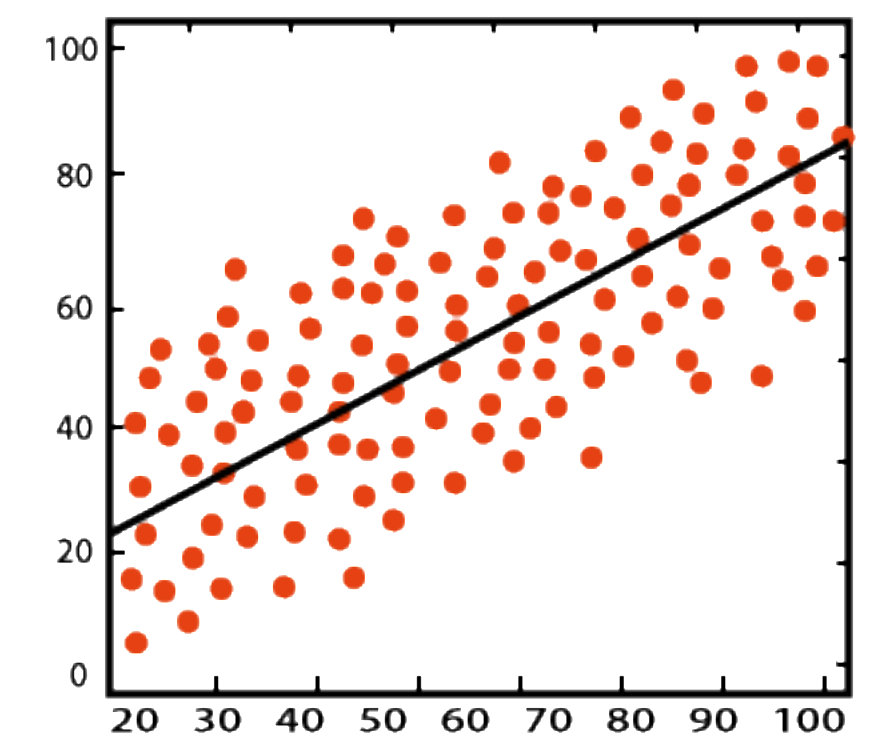
Review: Basic Machine Learning Terminology

labeled training data

- **Supervised** vs Unsupervised learning
- **Classification** vs Regression
- Discriminative vs **Generative** models
- We will do **Supervised Text Classification**

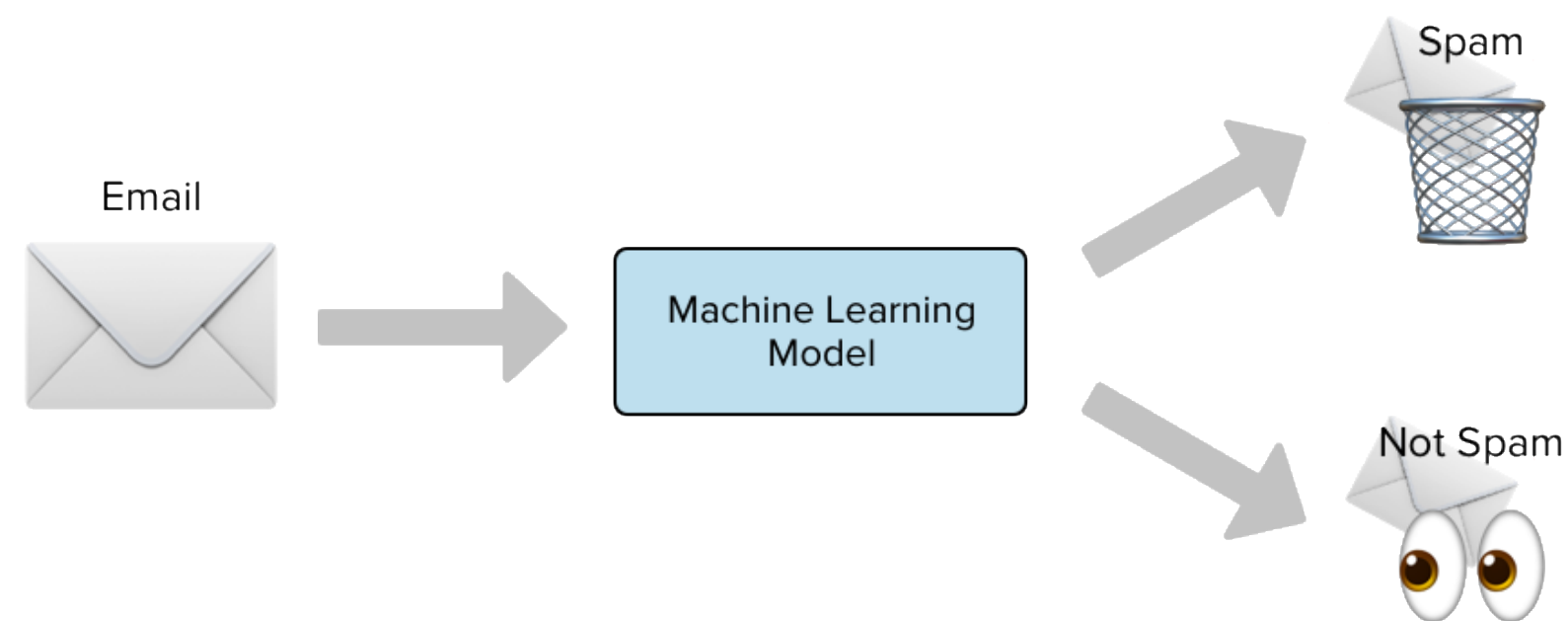


Classification

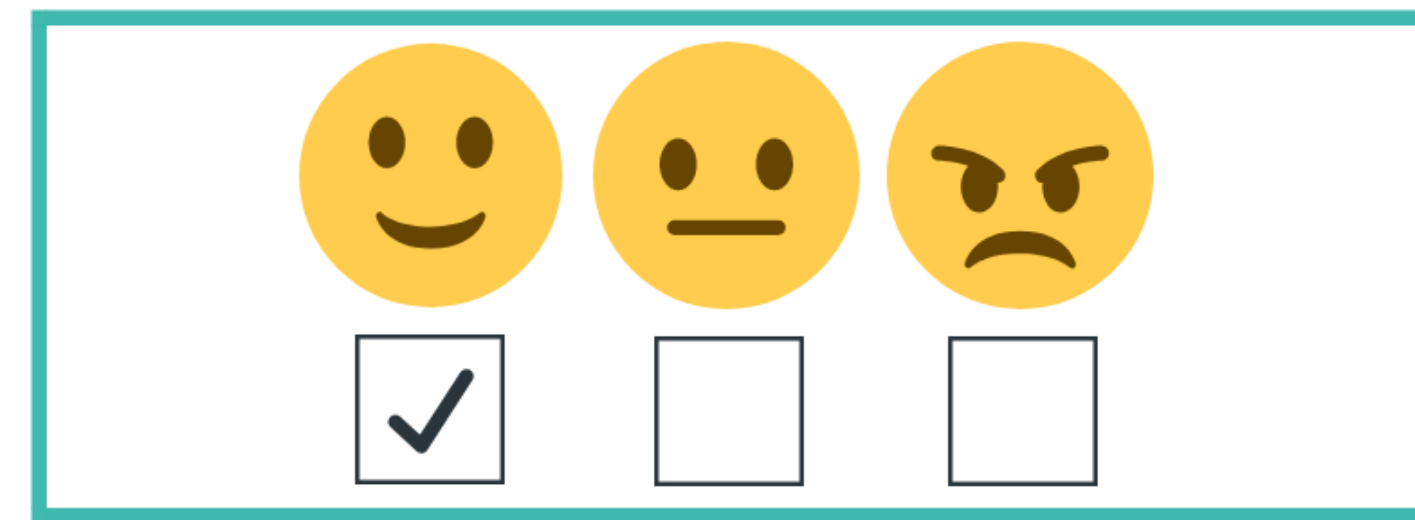


Regression

Why classify?



Spam detection



Sentiment analysis

Movie Reviews

neg: unbelievably disappointing

pos: Full of zany characters and richly applied satire, and some great plot twists

pos: this is the greatest screwball comedy ever filmed

neg: It was pathetic. The worst part about it was the boxing scenes.

- Authorship attribution
- Language detection
- News categorization

Classification as a subtask in NLP

- NLP is (mostly) about classification
 - Text classification: Spam/Not Spam, Sentiment Analysis
 - **Language modelling / generating sentences:** select word to generate at each step (classification over vocabulary!)
 - Building dialog system (identifying intent)
 - Parsing (identifying word to attach to)

Classification as a subtask in NLP

Intent detection

ADDR_CHANGE: I just moved and want to change my address.

ADDR_CHANGE: Please help me update my address

FILE_CLAIM: I just got into a terrible accident and I want to file a claim

CLOSE_ACCOUNT: I'm moving and I want to disconnect my service

Prepositional phrase attachment

noun attach: *I bought **the shirt** with pockets*

verb attach: *I **bought** the shirt with my credit card*

noun attach: *I washed **the shirt** with mud*

verb attach: *I **washed** the shirt with soap*

Text classification: the task

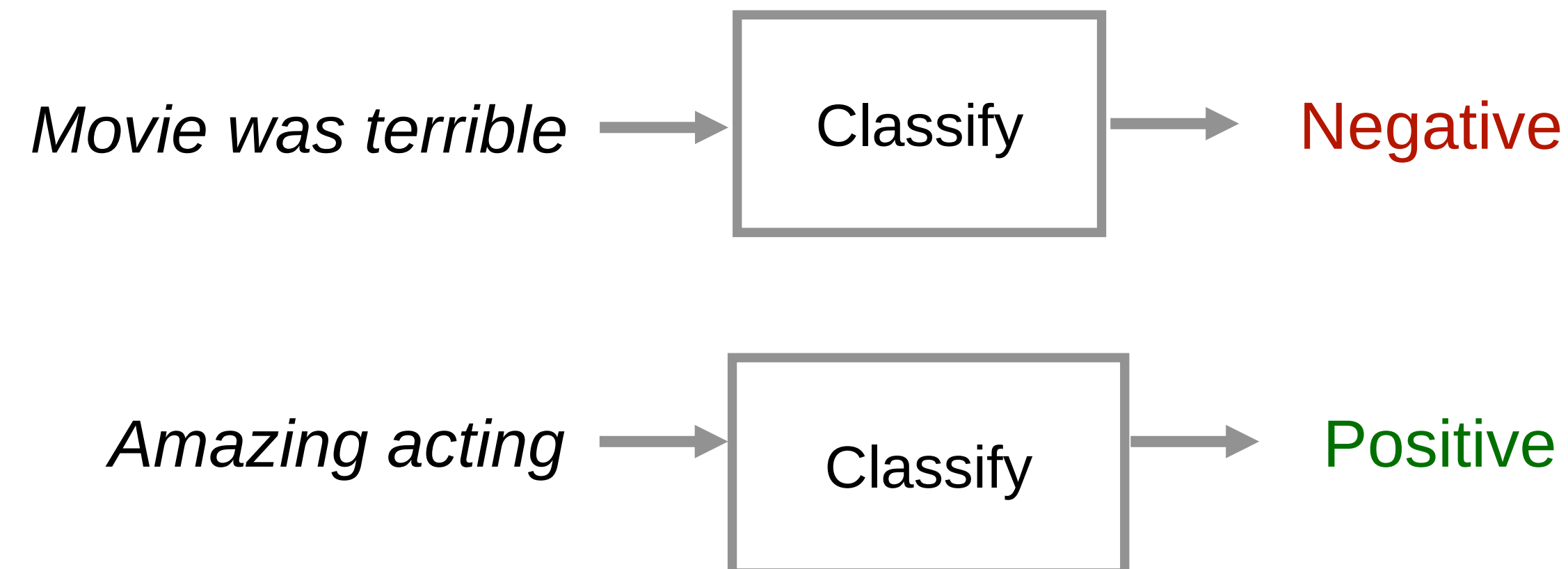
- Inputs:

- A document **d**
- A set of classes **C** = {**c**₁, **c**₂, **c**₃, ..., **c**_m}

Multiple classes: m
Binary: m=2

- Output:

- Predicted class **c** for document **d**



sequence of words
sentence

Rule-based classification

- Look for patterns, and combinations of features on words in document, meta-data

IF there exists word w in document d such that w in [good, great, extra-ordinary, ...],
THEN output Positive

IF email address ends in [ithelpdesk.com, makemoney.com, spinthewheel.com, ...]
THEN output SPAM

- Simple, can be very accurate
- But: rules may be hard to define (and some even unknown to us!)
 - Expensive
 - Not easily generalizable

Supervised Learning: Let's use statistics!

- Data-driven approach

Let the machine figure out the best patterns to use!

- Inputs:

- Set of m classes $C = \{c_1, c_2, \dots, c_m\}$
- Set of n 'labeled' documents: $\{(d_1, c_1), (d_2, c_2), \dots, (d_n, c_n)\}$

- Output: Trained classifier, $F: d \rightarrow c$

- What form should F take?
- How to learn F ?



Designing machine learning models

general recipe

- Input **features**: $f(x) \rightarrow [f_1, f_2, \dots, f_m]$

- Need to determine features

- Output: estimate $P(y|x)$ for each class c

Linear / Logistic Regression Family

Neural Network Family (MLP)

Naive Bayes Family

- Need to model $P(y|x)$ with a **family of functions** $\leftarrow \mathcal{H} = \{f_\theta(x) \mid \theta \in \Theta\}$

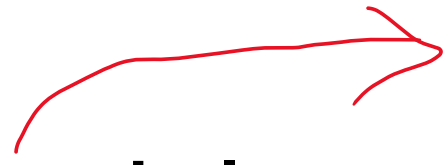
Building
the model

- Train phase: Learn **parameters** of model to minimize loss function

- Need **training objective** and **optimization** algorithm

- Test phase: Apply parameters to predict class given a new input

General guidelines for model building

Two steps to building a probability model:  a mathematical framework that describes a process by assigning valid probabilities (values in $[0, 1]$ that sum or integrate to 1) to events or sequences.

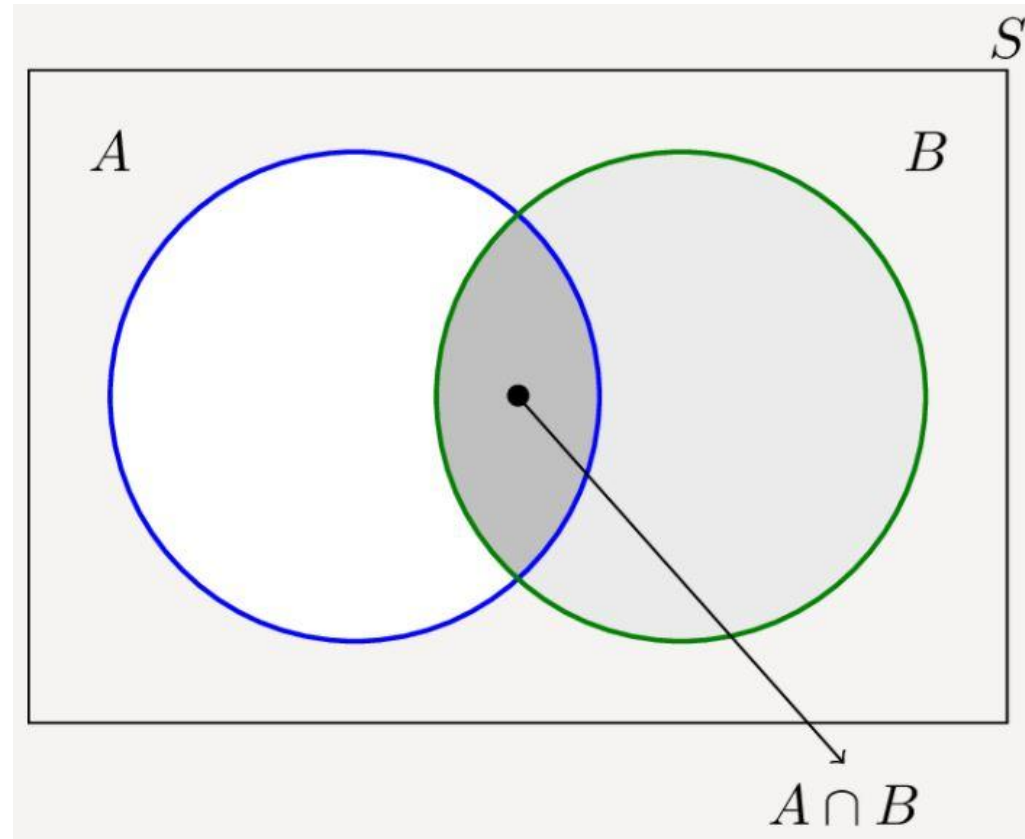
1. Define the model

- What form should F take?
- What independence assumptions do we make?
- What are the model parameters (probability values)?
For example, $P(\text{"apple"} \mid \text{"eat"}) = 0.04$

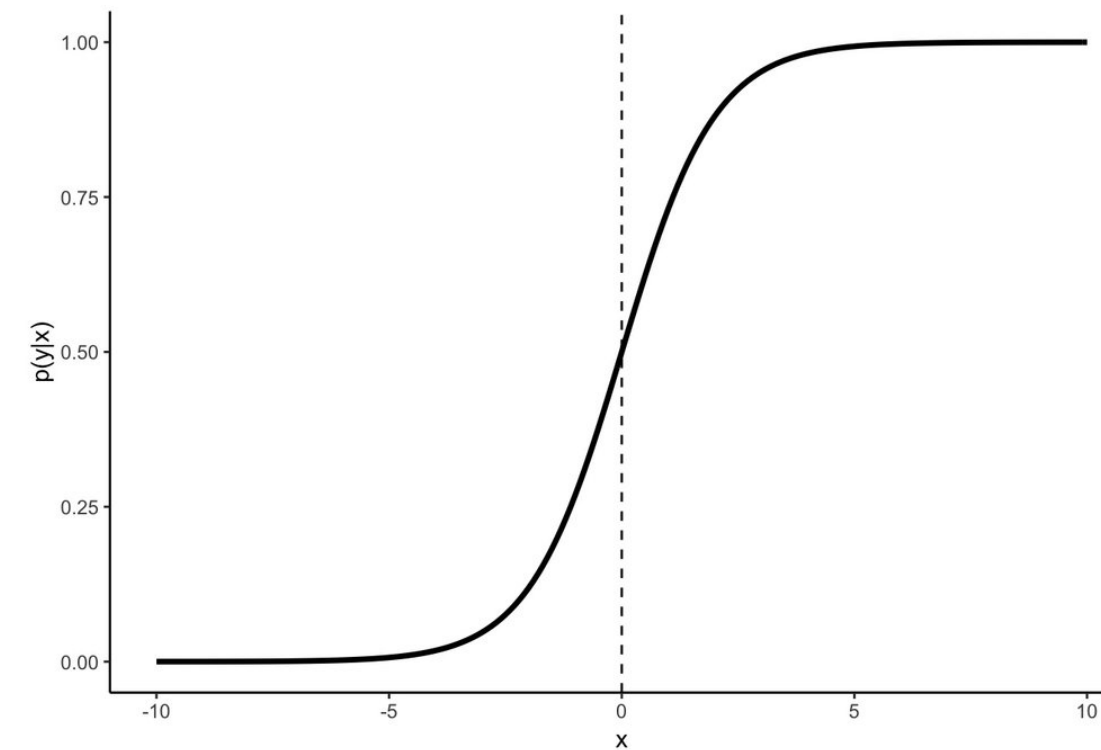
2. Estimate the model parameters (training/learning)

- How to learn F ? What to optimize? What is the training objective?

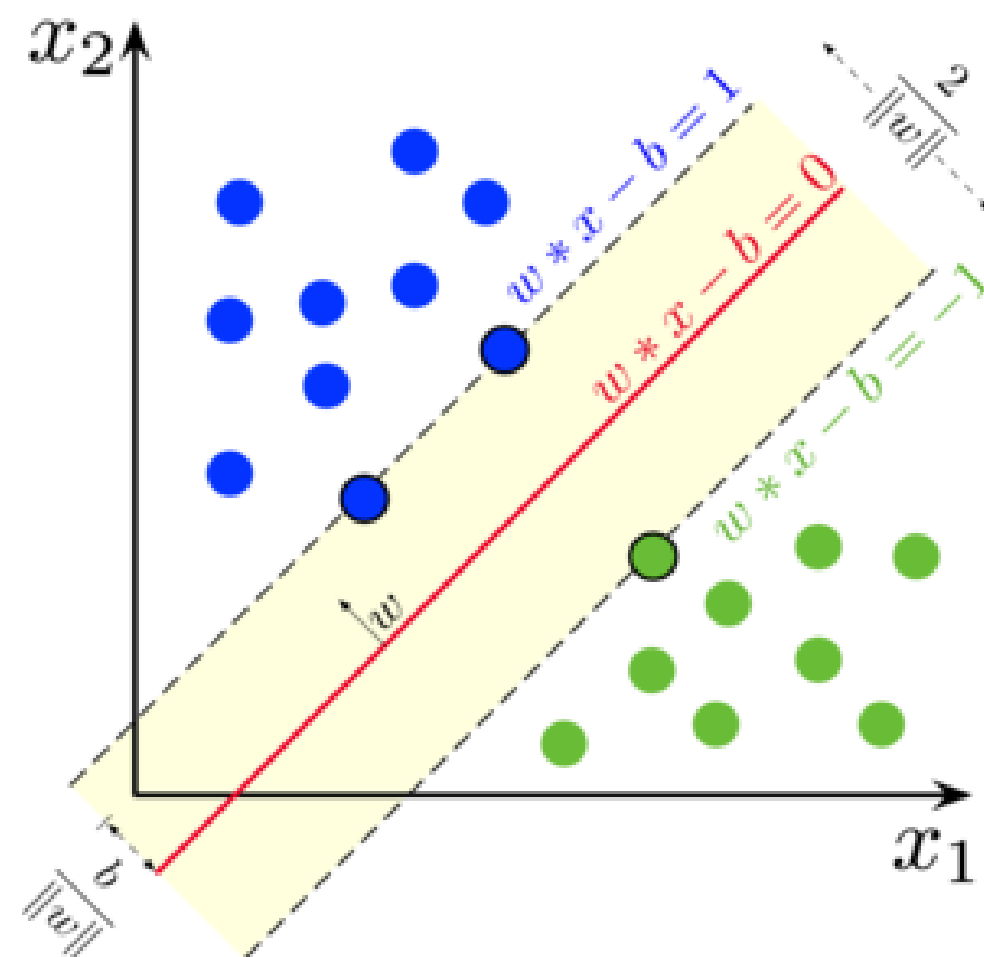
Types of supervised classifiers



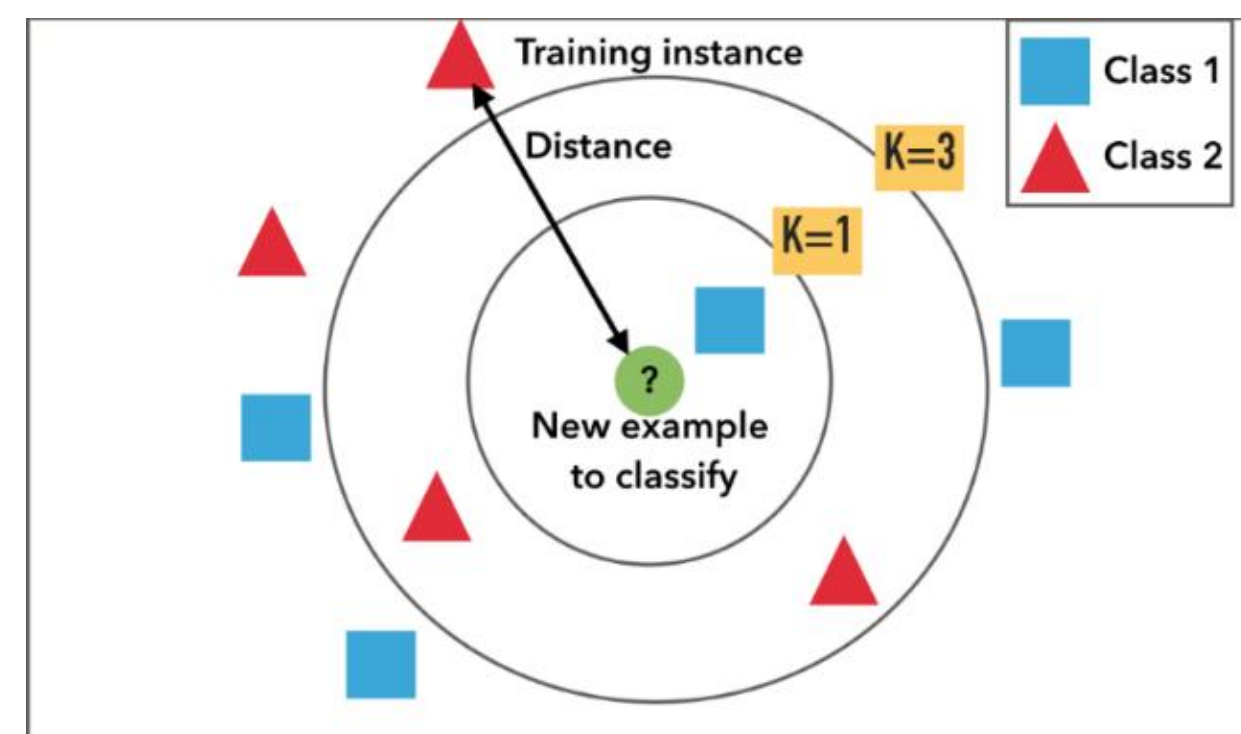
Naive Bayes



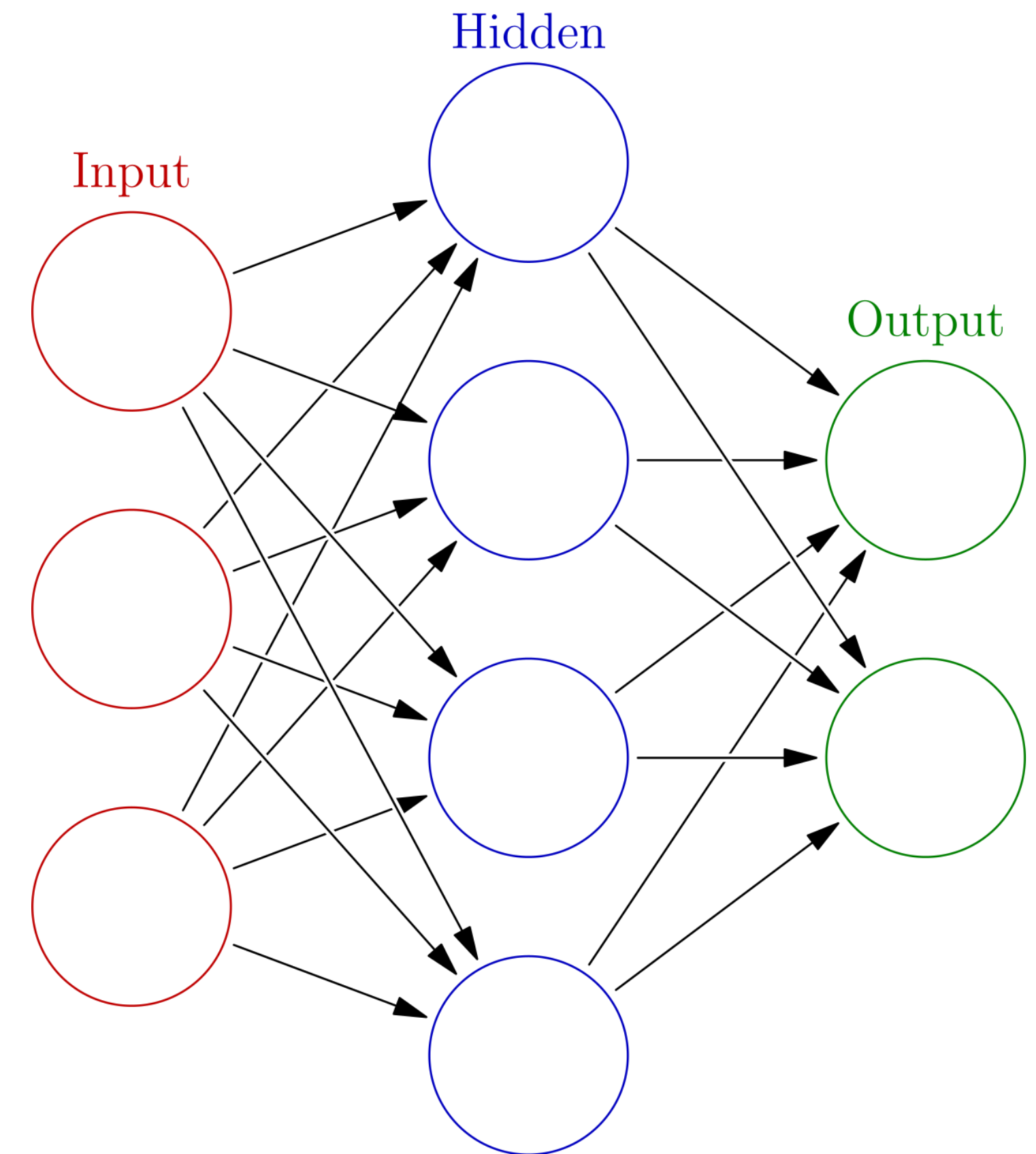
Logistic regression



Support vector machines



k-nearest neighbors



Neural networks

Naive Bayes

Naive Bayes Classifier

General setting

- Let the **input** x (e.g., two sentences) be represented as r features: $f_j, 1 \leq j \leq r$
- Let y be the output classification
- We can have a simple classification model using **Bayes rule**

$$\begin{array}{c} \text{Posterior} \\ P(y|x) = \frac{\overset{\text{Prior}}{P(y)} \cdot \overset{\text{Likelihood}}{P(x|y)}}{\underset{\text{Evidence}}{P(x)}} \end{array}$$

- Make strong (naive) **conditional independence assumptions**

$$P(x|y) = \prod_{j=1}^r P(f_j|y) \xrightarrow{\text{Bayes rule}} P(y|x) \propto P(y) \cdot \prod_{j=1}^r P(f_j|y)$$

$P(x)$ is constant. Remove it.

Naive Bayes classifier for text classification

- For text classification: input x is document $d = (w_1, \dots, w_k)$
- Use as our features the words w_j , $1 \leq j \leq |V|$ where V is our vocabulary
- c is the output classification
- Predicting the best class:

maximum a posteriori
(MAP) estimate

↗ C_{MAP}

$$\begin{aligned}
 &= \operatorname{argmax}_{c \in C} P(c|d) \\
 &= \operatorname{argmax}_{c \in C} \frac{P(c)P(d|c)}{P(d)} \\
 &= \operatorname{argmax}_{c \in C} P(c)P(d|c)
 \end{aligned}$$

$$P(y|x) = \frac{P(y) \cdot P(x|y)}{P(x)}$$

$P(d|c)$ → Conditional probability of generating document d from class c

$P(c)$ → Prior probability of class c

Represent $P(d \mid c)$ as Bag of Words model

- Assume position of each word is irrelevant (both absolute and relative) Order doesn't matter
- $P(w_1, w_2, w_3, \dots, w_k \mid c) = P(w_1 \mid c)P(w_2 \mid c) \dots P(w_k \mid c)$
- Probability of each word is conditionally independent given class c

the
cat
the
sat
mat
uo



Bag of words

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



word	count
it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Predicting with Naive Bayes

- Once we assume that the position of each word is irrelevant and that the words are **conditionally independent** given class c , we have:

$$P(d|c) = P(w_1, w_2, w_3, \dots, w_k|c) = P(w_1|c)P(w_2|c) \dots P(w_k|c)$$

- The **maximum a posteriori** (MAP) estimate is now:

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(c)P(d|c) = \underset{c \in C}{\operatorname{argmax}} \hat{P}(c) \prod_{i=1}^k \hat{P}(w_i|c)$$

$\hat{P}$ is used to indicate the estimated probability

Prior probability

Note that k is the number of tokens (words) in the document.

The index i is the position of the token.

$P(d|c) \rightarrow$ Conditional probability of generating document d from class c

Maximum likelihood estimate

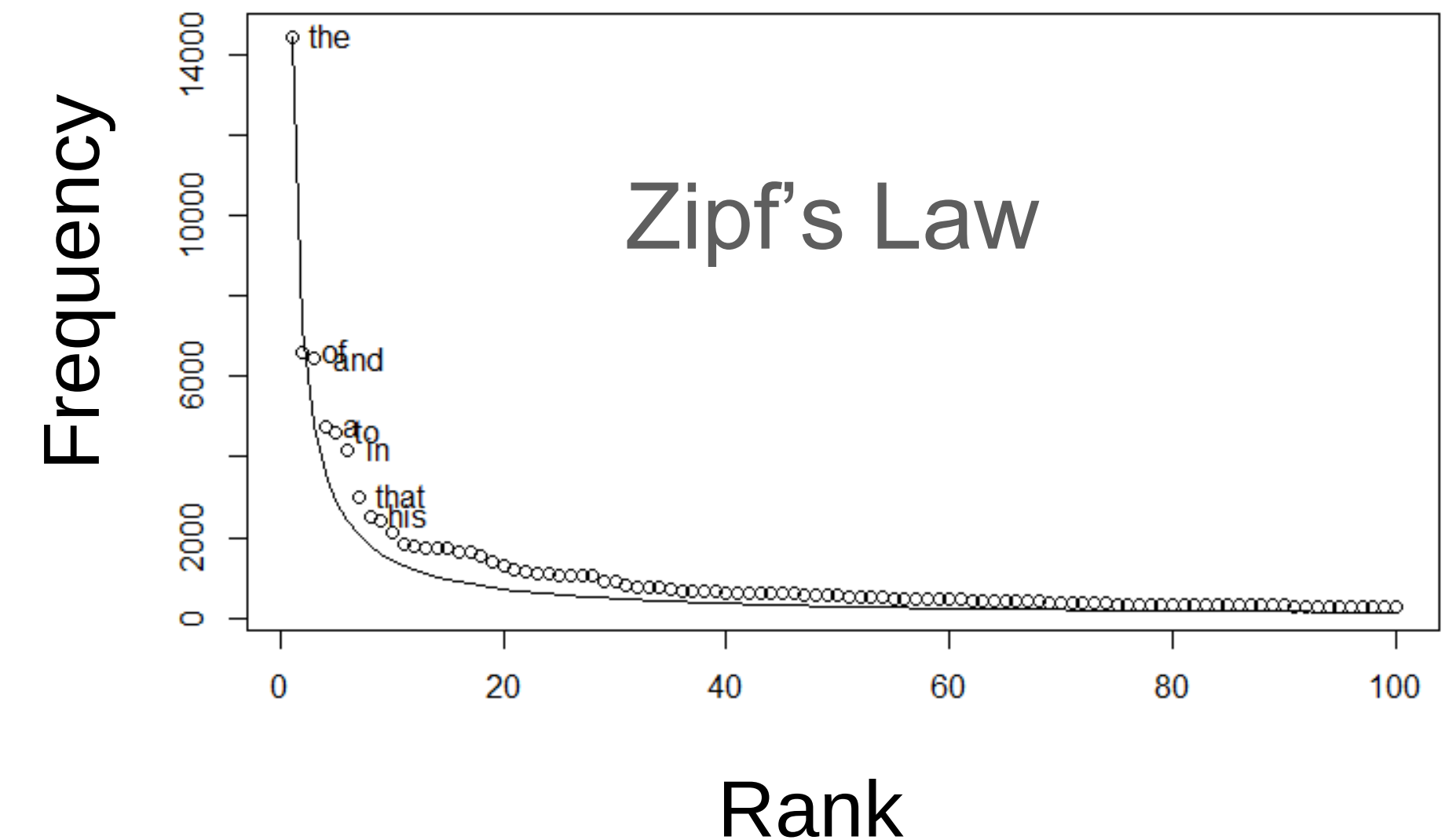
- Count and take average:

$$\hat{P}(c_j) = \frac{\text{Count}(c_j)}{n}$$

$$\hat{P}(w_i|c_j) = \frac{\text{Count}(w_i, c_j)}{\sum_{w \in V} [\text{Count}(w, c_j)]}$$

total number of times the word w occurs
across all training documents labeled with class c_j

Can suffer from sparsity issues!



Suppose training on review classification with class $c_j = \text{"Positive"}$:

- The training set has two positive documents:
 - Doc 1: "good food good service"
 - Doc 2: "good price"
- $\text{Count}(\text{"good"}, \text{"Positive"}) = 2 + 1 = \mathbf{3}$
- $\sum_{w \in V} \text{Count}(w, \text{"Positive"}) = 4 + 2 = \mathbf{6}$ (total tokens)
- $\hat{P}(\text{"good"} \mid \text{"Positive"}) = \frac{3}{6} = 0.5$

Data sparsity

- What about when $\text{count}(\text{'amazing'}, \text{positive}) = 0$?
- Implies $P(\text{'amazing'} \mid \text{positive}) = 0$
- Given a review document, $d = \text{".... most amazing movie ever ..."}$

$$C_{\text{MAP}} = \arg \max_{c \in C} \hat{p}(c) \prod_{i=1}^k P(w_i \mid c)$$

$$= \arg \max_{c \in C} \hat{p}(c) \cdot 0 = \arg \max_{c \in C} 0$$

Can't determine the best c !

Solution: Smoothing!

- Maximum likelihood estimate

$$\hat{P}(c_j) = \frac{\text{Count}(c_j)}{n}$$

$$\hat{P}(w_i|c_j) = \frac{\text{Count}(w_i, c_j)}{\sum_{w \in V} [\text{Count}(w, c_j)]}$$

- Smoothing

$$\hat{P}(w_i|c_j) = \frac{\text{Count}(w_i, c_j) + \alpha}{\sum_{w \in V} [\text{Count}(w, c_j) + \alpha]}$$

Laplace smoothing

- Simple, easy to use
- Effective in practice

a posteriori is Latin for "from what comes after"

Overall process

-Hindsight is 20/20?
-- Often times used in interview

- Input: Set of annotated documents $\{(d_i, c_i)\}_{i=1}^n$

A. Compute vocabulary **V** of all words

B. Calculate
$$\hat{P}(c_j) = \frac{\text{Count}(c_j)}{n}$$

C. Calculate
$$\hat{P}(w_i|c_j) = \frac{\text{Count}(w_i, c_j) + \alpha}{\sum_{w \in V} [\text{Count}(w, c_j) + \alpha]}$$

D. (Prediction) Given document $d = (w_1, w_2, \dots, w_k)$

What you believe about the class *c* after observing document *d*

$$c^* = \arg \max_c P(c | d)$$

$$P(c | d) = \frac{P(d | c)P(c)}{P(d)}$$

$$P(d | c) = P(w_1, w_2, \dots, w_k | c) \approx \prod_{i=1}^k P(w_i | c)$$

Also see P14

$$c_{\text{MAP}} = \arg \max_c \hat{P}(c) \prod_{i=1}^k \hat{P}(w_i|c)$$

Predicted class

to pick the class with the highest probability given the observed data

how many times a word occurs in a document

Variants

the output classification

$$P(f_i|y) \rightarrow P(w_i|c)$$

Name based on the distribution of the features

$$\hat{P}(c_j) = \frac{\text{Count}(c_j)}{n}$$

n documents

Multinomial Naive Bayes

Normal counts (0,1,2,...) for each document

Some work show this works better than full counts or the Multivariate Bernoulli NB

Binary Multinomial NB

Binarized counts (0/1) for each document

the model represents each document as a vector of **multiple random variables** considered simultaneously

Multivariate Bernoulli NB

Estimate $P(w|c)$ as fraction of documents of class c with word w

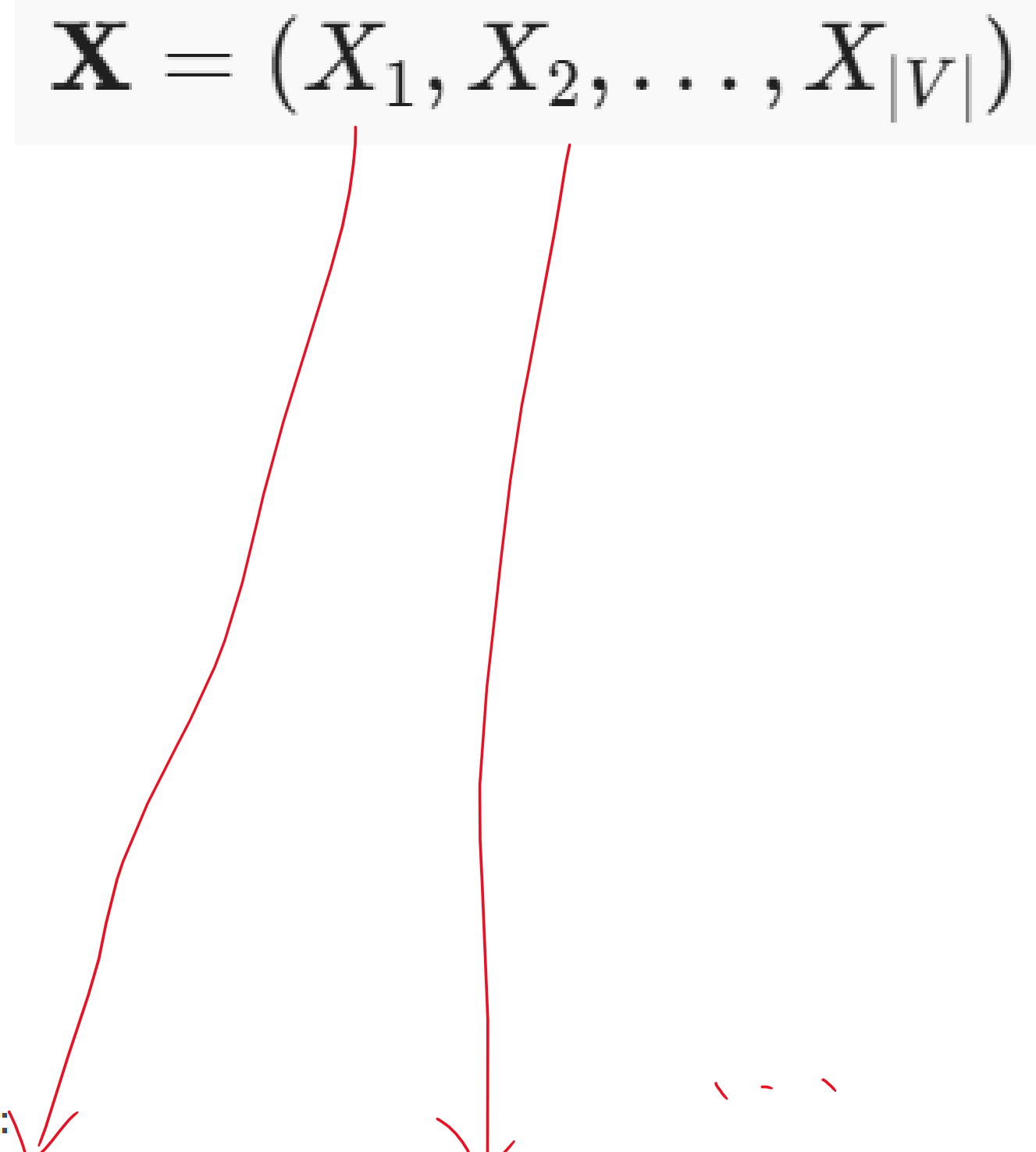
- Explicitly model $P(!w|c) = 1 - P(w|c)$

E.g. : A mini spam filter.

Word (w)	$P(w SPAM)$	$P(!w SPAM)=1-P(w SPAM)$	$P(w HAM)$	$P(!w HAM)=1-P(w HAM)$
"viagra"	0.80	0.20	0.01	0.99
"lottery"	0.70	0.30	0.05	0.95
"meeting"	0.10	0.90	0.60	0.40
"lunch"	0.05	0.95	0.50	0.50

Viagra occur 80 times, in 100 SPAM emails.

Viagra occur 1 time, in 100 HAM emails.

$$\mathbf{X} = (X_1, X_2, \dots, X_{|V|})$$


- Likelihood given SPAM:

$$P(d | SPAM) = P(\text{"meeting"} | SPAM) \cdot P(\text{"viagra"} | SPAM) \cdot P(\text{"lottery"} | SPAM) \cdot P(\text{"lunch"} | SPAM)$$

$$P(d | SPAM) = 0.10 \times 0.20 \times 0.30 \times 0.95 = 0.0057$$

- Likelihood given HAM:

$$P(d | HAM) = P(\text{"meeting"} | HAM) \cdot P(\text{"viagra"} | HAM) \cdot P(\text{"lottery"} | HAM) \cdot P(\text{"lunch"} | HAM)$$

$$P(d | HAM) = 0.60 \times 0.99 \times 0.95 \times 0.50 \approx 0.2821$$

Naive Bayes Example

$$\hat{P}(c) = \frac{N_c}{N}$$

Smoothing with $\alpha = 1$

$$\hat{P}(w | c) = \frac{\text{count}(w, c) + 1}{\text{count}(c) + |V|}$$

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

Priors:

$$P(c) =$$

$$P(j) =$$

Choosing a class:

$$P(c | d_5) \propto$$

Conditional Probabilities:

$$P(\text{Chinese} | c) =$$

$$P(\text{Tokyo} | c) =$$

$$P(\text{Japan} | c) =$$

$$P(\text{Chinese} | j) =$$

$$P(\text{Tokyo} | j) =$$

$$P(\text{Japan} | j) =$$

$$P(j | d_5) \propto$$

(Credits: Dan Jurafsky)

Naive Bayes Example

$$\hat{P}(c) = \frac{N_c}{N}$$

Smoothing with $\alpha = 1$

$$\hat{P}(w|c) = \frac{\text{count}(w,c) + 1}{\text{count}(c) + |V|}$$

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

- Let's compute the priors: what is $\hat{P}(c)$ and $\hat{P}(j)$? $\hat{P}(c) = \frac{3}{4}, \hat{P}(j) = \frac{1}{4}$

- Let's compute $\hat{P}(Japan|c)$:

$$\text{count}(Japan, c) = 0 \quad \text{count}(c) = \sum_{w \in V} \text{count}(w, c) = 8 \quad |V| = 6$$

How many words in the documents of c class

$$\hat{P}(Japan|c) = \frac{\text{count}(Japan, c) + 1}{\text{count}(c) + |V|}$$

(Credits: Dan Jurafsky)

Naive Bayes Example

$$\hat{P}(c) = \frac{N_c}{N}$$

Smoothing with $\alpha = 1$

$$\hat{P}(w | c) = \frac{\text{count}(w, c) + 1}{\text{count}(c) + |V|}$$

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

Priors:

$$P(c) =$$

$$P(j) =$$

Choosing a class:

$$P(c | d5) \propto$$

Conditional Probabilities:

$$P(\text{Chinese} | c) =$$

$$P(\text{Tokyo} | c) =$$

$$P(\text{Japan} | c) =$$

$$P(\text{Chinese} | j) =$$

$$P(\text{Tokyo} | j) =$$

$$P(\text{Japan} | j) =$$

$$P(j | d5) \propto$$

(Credits: Dan Jurafsky)

Some details

- **Vocabulary** is important
- **Tokenization** matters: it can affect your **vocabulary**
 - Tokenization = how you break your sentence up into tokens / words
 - Make sure you are consistent with your tokenization!

▶ *aren't* aren't
arent
are n't
aren t

▶ Emails, URLs, phone numbers, dates, emoticons

- Special multi-word tokens: NOT_happy
- Modern NLP system use **subword tokens** (e.g. byte pair encoding)

Some details

- **Vocabulary** is important
- **Tokenization** matters: it can affect your **vocabulary**
 - Tokenization = how you break your sentence up into tokens / words
 - Make sure you are consistent with your tokenization!
- Handling **unknown** words in test not in your training vocabulary?
 - Remove them from your test document! Just ignore them.
- Handling **stop** words (common words like *a*, *the* that may not be useful)
 - Remove them from the training data!
 - In practice not that helpful, so use all words!

Better to use

- Modified counts (tf-idf) that down weighs frequent, unimportant words
- **Better models!**

term frequency–inverse document frequency

Features

- In general, Naive Bayes can use **any set of features**, not just words
- URLs, email addresses, Capitalization, ...
- Domain knowledge can be crucial to performance

*Top features
for
Spam detection*

Rank	Category	Feature	Rank	Category	Feature
1	Subject	Number of capitalized words	1	Subject	Min of the compression ratio for the bz2 compressor
2	Subject	Sum of all the character lengths of words	2	Subject	Min of the compression ratio for the zlib compressor
3	Subject	Number of words containing letters and numbers	3	Subject	Min of character diversity of each word
4	Subject	Max of ratio of digit characters to all characters of each word	4	Subject	Min of the compression ratio for the lzw compressor
5	Header	Hour of day when email was sent	5	Subject	Max of the character lengths of words
(a)			(b)		
Spam URLs Features					
1	URL	The number of all URLs in an email	1	Header	Day of week when email was sent
2	URL	The number of unique URLs in an email	2	Payload	Number of characters
3	Payload	Number of words containing letters and numbers	3	Payload	Sum of all the character lengths of words
4	Payload	Min of the compression ratio for the bz2 compressor	4	Header	Minute of hour when email was sent
5	Payload	Number of words containing only letters	5	Header	Hour of day when email was sent

Properties of Naive Bayes

- + Simple baseline method
- + Works well for small data sizes
- + Optimal if the independence assumptions hold: if the assumed independence is correct, then it is the Bayes Optimal Classifier for the problem
- But not if the independence assumption is broken
- Does not handle rare classes well - will favour more common class
- Also need to design features
- **Modern NLP: use large neural language models with learned representations**

Common words remain single tokens: "the", "cat", "apple".

Rare, compound, or newly invented words are split into recognizable, reusable subword chunks:

○ "unhappily" → ["un", "happi", "ly"]

Generative vs Discriminative Models

- Naive Bayes is a Generative Model: It models $p(y|x) \propto p(y)p(x|y)$
- It models how the document is generated from words
- You can use this model to sample documents
- Next: Logistic Regression, a Discriminative model that models $p(y|x)$ directly.

Generating an Artificial Document:

1. Roll a 2-sided coin to choose the class y :
Suppose the outcome is $y = \text{Sports}$.
2. Fix the document length:
Suppose we want a 4-word document ($L = 4$).
3. Roll a loaded 5-sided die with the Sports distribution 4 times:

Word probabilities given Sports:

- $P(\text{"game"} \mid \text{Sports}) = 0.40$
- $P(\text{"score"} \mid \text{Sports}) = 0.30$
- $P(\text{"ball"} \mid \text{Sports}) = 0.20$
- $P(\text{"code"} \mid \text{Sports}) = 0.05$
- $P(\text{"app"} \mid \text{Sports}) = 0.05$

- $w_i \sim P(w \mid y)$
- **biased**
Draw 1: Roll lands in the top 40% interval → "game"
 - Draw 2: Roll lands in the 40% – 70% interval → "score"
 - Draw 3: Roll lands in the 70% – 90% interval → "ball"
 - Draw 4: Roll lands in the top 40% interval again → "game"

Generated Document:

: "game score ball game" (Label: Sports)

Evaluation

Evaluation

- Consider binary classification
- Table of predictions

Confusion matrix



		<i>Truth</i>	
		Positive	Negative
<i>Predicted</i>	Positive	100	5
	Negative	45	100

- Ideally, we want:

		<i>Truth</i>	
		Positive	Negative
<i>Predicted</i>	Positive	145	0
	Negative	0	105

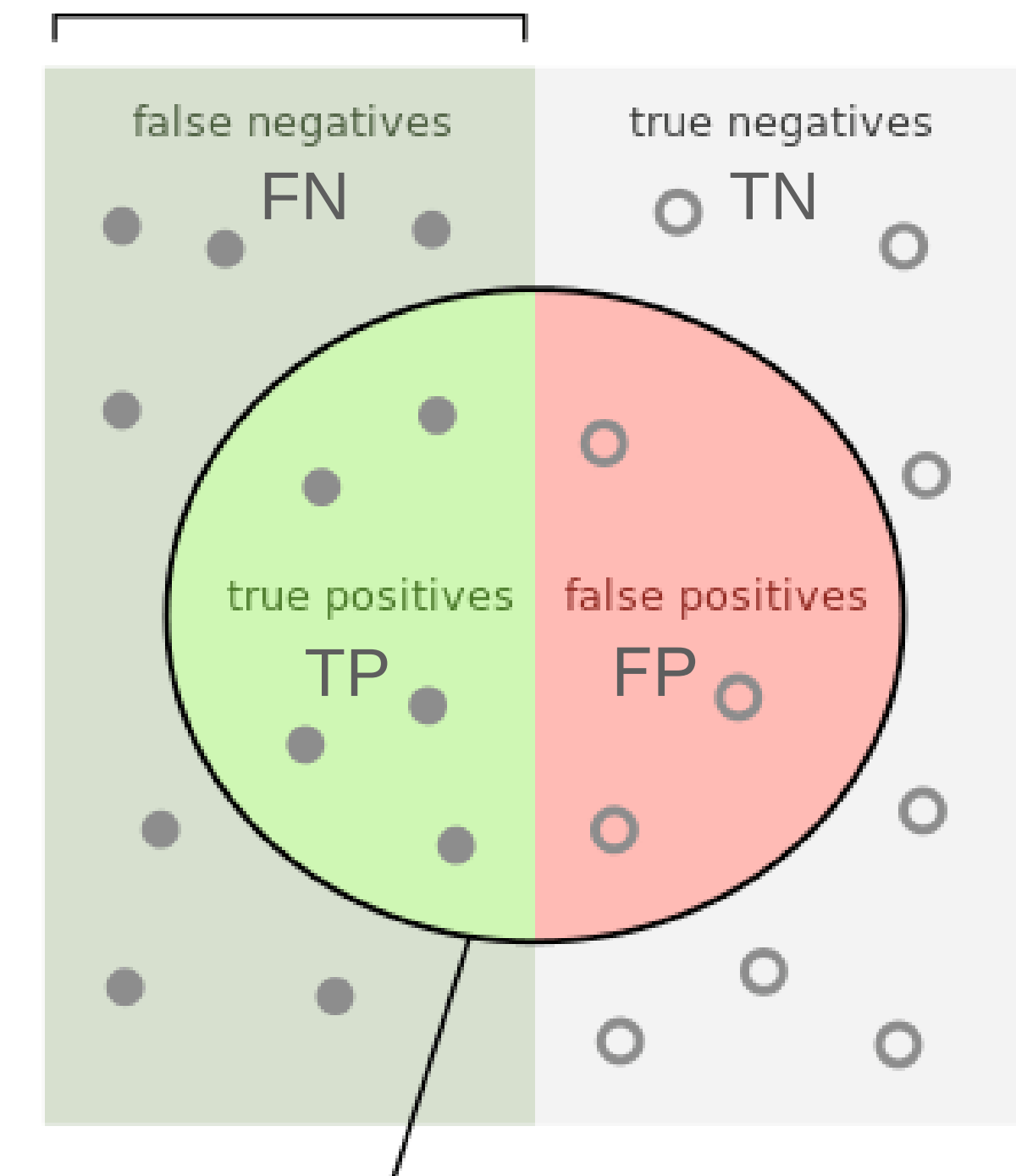
Evaluation Metrics

Confusion matrix

		Truth	
		Positive	Negative
Predicted	Positive	100 TP	5 FP
	Negative	45 FN	100 TN

- True positive (TP): Predicted + and actual +
- True negative (TN): Predicted - and actual -
- False positive (FP): Predicted + and actual -
- False negative (FN): Predicted - and actual +

Actual positives



Predicted positives

(image credit: wikipedia)

$$Accuracy = \frac{TP + TN}{Total} = \frac{200}{250} = 80\%$$

Coarse metric

Evaluation Metrics

		<i>Truth</i>	
		Positive	Negative
<i>Predicted</i>	Positive	100	5
	Negative	45	100

		Positive	Negative
	Positive	50	25
	Negative	25	150

- True positive (TP): Predicted + and actual +
- True negative (TN): Predicted - and actual -
- False positive (FP): Predicted + and actual -
- False negative (FN): Predicted - and actual +

Accuracy cannot distinguish
between the two models!

$$Accuracy = \frac{TP + TN}{Total} = \frac{200}{250} = 80\%$$

Precision and Recall

- Precision: % of selected classes that are correct

To find the relevant items

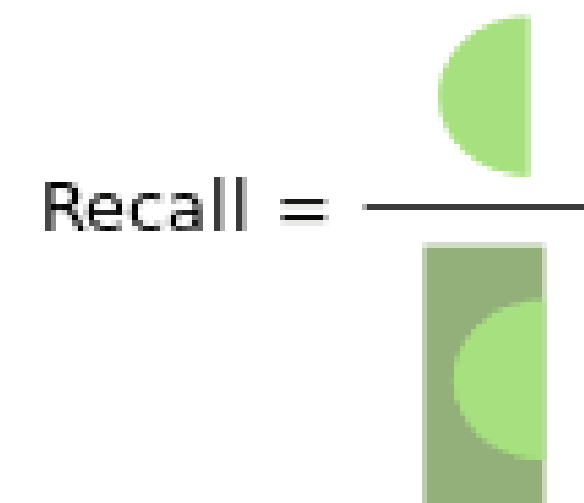
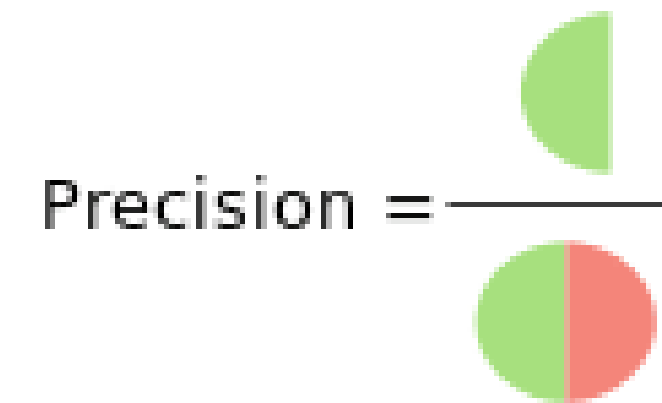
$$\text{Precision}(+) = \frac{TP}{TP + FP}$$

Out of everything the system returned, **how accurate is it**

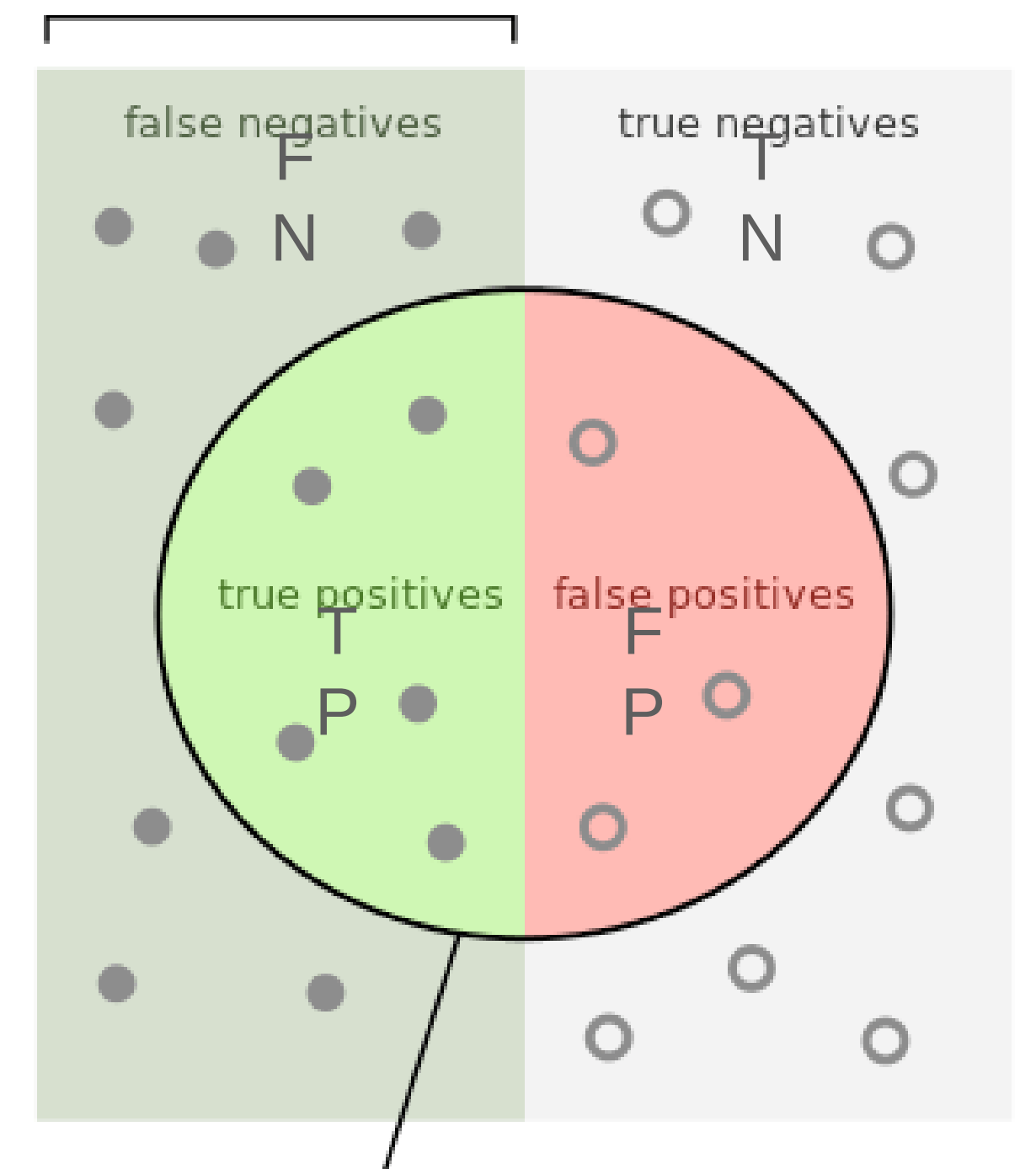
- Recall: % of correct items selected

$$\text{Recall}(+) = \frac{TP}{TP + FN}$$

Relevant items successfully fetched back
All relevant items that exist in reality



Actual positives(relevant)



Predicted positives
(selected/retrieved)

(image credit: wikipedia)

Evaluation Metrics

		Truth	
		Positive	Negative
Predicted	Positive	100	5
	Negative	45	100

		Positive	Negative
		Positive	Negative
Predicted	Positive	50	25
	Negative	25	150

$Precision(+) = \frac{TP}{TP + FP}$

$\frac{100}{100 + 5} = 0.95$

$\frac{50}{50 + 25} = 0.75$

$Recall(+) = \frac{TP}{TP + FN}$

$\frac{100}{100 + 45} = 0.69$

$\frac{50}{50 + 25} = 0.75$

Two metrics - which one to use?

F-Score

- Combined measure
- Harmonic mean of Precision and Recall

$$F_1 = \frac{2 \cdot \textit{Precision} \cdot \textit{Recall}}{\textit{Precision} + \textit{Recall}}$$

- Or more generally,
- Use β to control importance of Precision vs Recall


$$F_\beta = \frac{(1 + \beta^2) \cdot \textit{Precision} \cdot \textit{Recall}}{\beta^2 \cdot \textit{Precision} + \textit{Recall}}$$

Evaluation Metrics

		Truth	
Predicted		Positive	Negative
	Positive	100	5
	Negative	45	100

		Truth	
Predicted		Positive	Negative
	Positive	50	25
	Negative	25	150

$$Precision(+) = \frac{TP}{TP + FP}$$

$$\frac{100}{100 + 5} = 0.95$$

$$\frac{50}{50 + 25} = 0.75$$

$$Recall(+) = \frac{TP}{TP + FN}$$

$$\frac{100}{100 + 45} = 0.69$$

$$\frac{50}{50 + 25} = 0.75$$

$$F_1(+) = \frac{2 \cdot P(+)R(+)}{P(+) + R(+)}$$

0.8

0.75

Aggregating scores

- How to handle more than 2 classes?
- We have Precision, Recall, F1 for each class

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	precision_u = $\frac{8}{8+10+1}$
	normal	5	60	50	precision_n = $\frac{60}{5+60+50}$
	spam	3	30	200	precision_s = $\frac{200}{3+30+200}$
		recall_u = $\frac{8}{8+5+3}$	recall_n = $\frac{60}{10+60+30}$	recall_s = $\frac{200}{1+50+200}$	

(Credits: Dan Jurafsky)

Aggregating scores

- How to handle more than 2 classes?
- We have Precision, Recall, F1 for each class
- How to combine them for an overall score?
- **Macro-average**: Compute for each class, then average
- **Micro-average**: Collect predictions for all classes and jointly evaluate

Macro vs Micro average

- Micro-averaged score is dominated by score on **common** classes

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

$$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = .60$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = .73$$

(Credits: Dan Jurafsky)

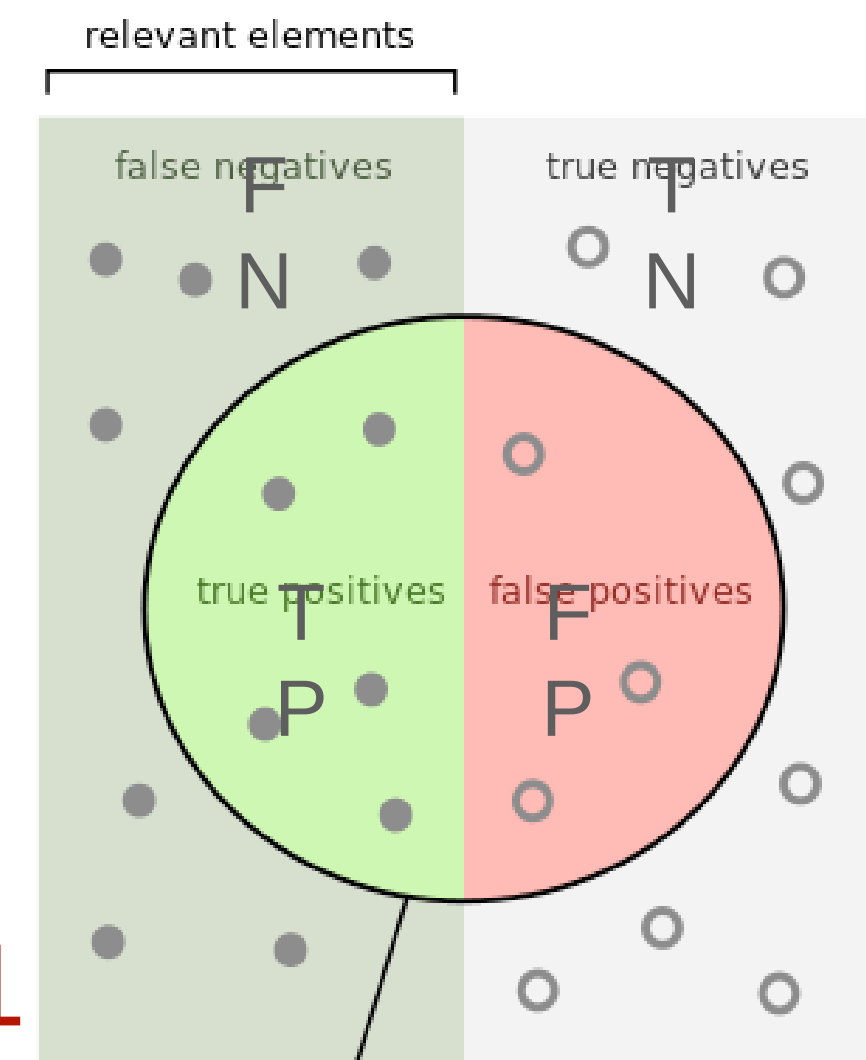
$$Precision(+)=\frac{TP}{TP+FP}$$

$$Recall(+)=\frac{TP}{TP+FN}$$

Precision Recall tradeoff

By changing the decision boundary, can tune the tradeoff.

Actual positives



Maximum F1

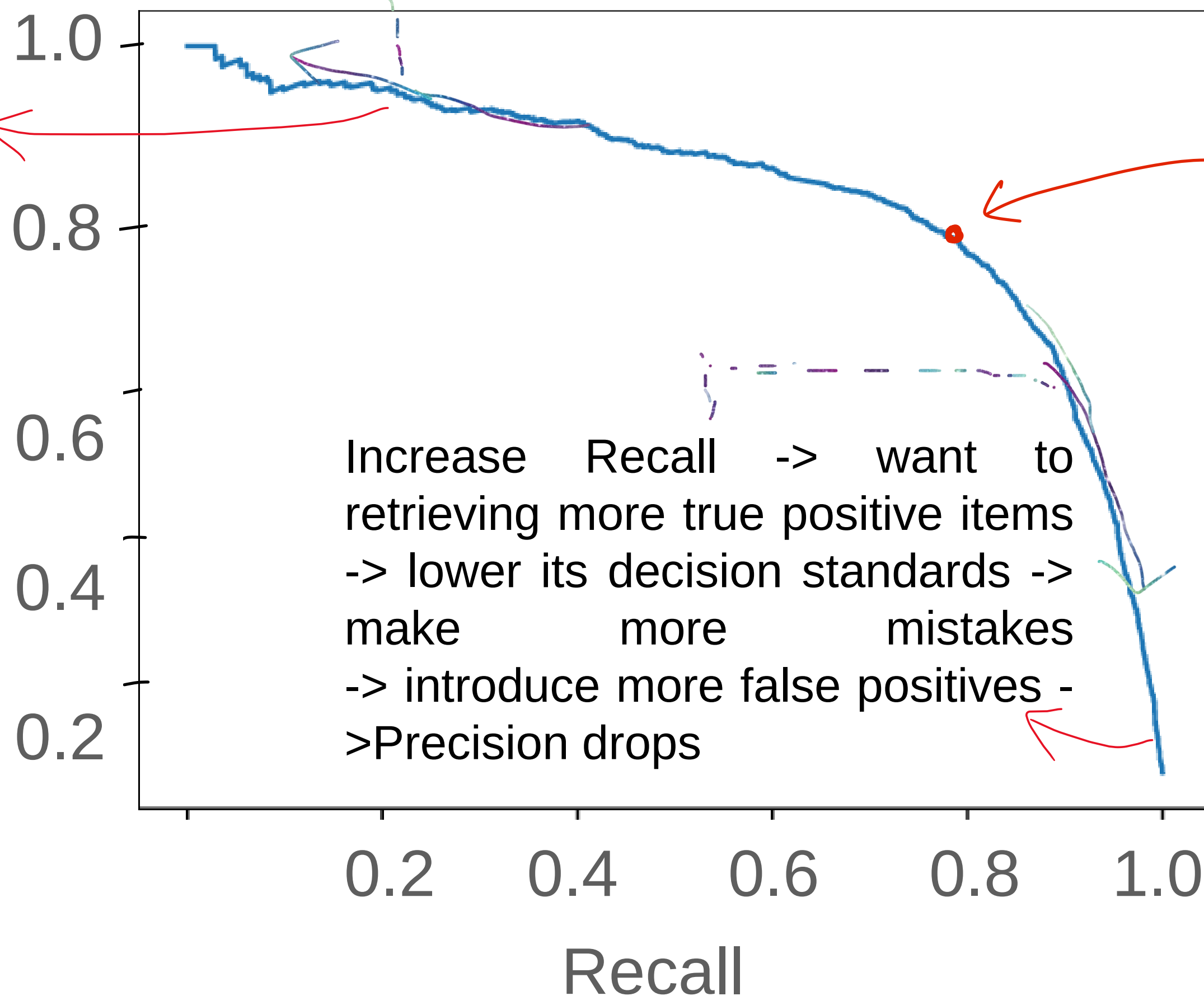
Predicted positives

Vary hyperparameters

- Smoothing α
- Threshold T

$$\frac{P(+|d)}{P(-|d)} > T$$

Tune on validation set



Vary hyperparameters

- Smoothing α
- Threshold T

$$\frac{P(+|d)}{P(-|d)} > T$$

Tune on validation set

A classifier usually outputs a probability or likelihood ratio rather than a hard label:

$$\frac{P(+|d)}{P(-|d)} > T$$

A sample

- **Default setting** ($T = 1$): Classify as positive if it's more likely than negative ($P(+|d) > P(-|d)$).
- **High threshold** ($T \gg 1$): The model only predicts positive when it is overwhelmingly confident.
 - Result: **High Precision, Low Recall** (few false alarms, but misses many actual positives).
- **Low threshold** ($T \ll 1$): The model predicts positive even with weak evidence.
 - Result: **High Recall, Low Precision** (finds almost all positives, but with lots of false alarms).

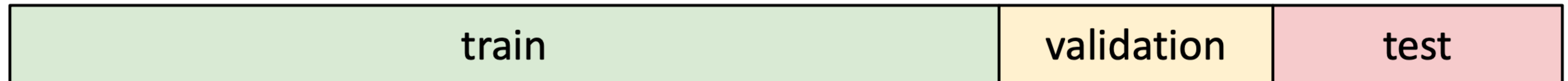
Sweeping T from ∞ down to 0 traces out the blue curve from top-left to bottom-right.

Smoothing parameter α :

Laplace smoothing parameter that adjusts probability estimates to prevent zero-counts and calibrate confidence.

Train, val, test split

- Train model on **training** set
- Tune hyperparameters on **validation** set
- Evaluate performance on unseen **test** set



Summary

- Evaluation Metrics
 - **Accuracy** - coarse metric
 - **Precision**, **Recall**, **F1** for each class
- Aggregated scores
 - **Macro-average**: Compute for each class, then average
 - **Micro-average**: Collect predictions for all classes and jointly evaluate (**dominated by common classes**)
- Precision-Recall curve: pick threshold for maximum F1
- Use **validation** set to tune hyperparameters, **test** set should remain “unseen”